# Ciclos `while` e Iteração Numérica

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

**Fundamentos de Programação**
2026–2027

Baseado em slides de Luís Caires

# Outline

1. Introdução à repetição
2. Contadores e acumuladores
3. Ciclos controlados pelos valores
4. Fibonacci como exemplo de estado
5. Raciocinar e testar ciclos

# Quando é Preciso Repetir

**Sem um ciclo:**

```
1 print(1)
2 print(2)
3 print(3)
4 print(4)
5 print(5)
```

# Quando é Preciso Repetir

**Sem um ciclo:**

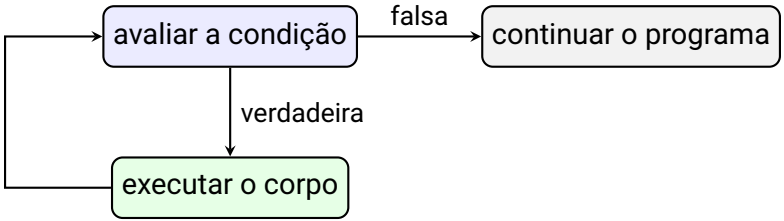
```
1 print(1)
2 print(2)
3 print(3)
4 print(4)
5 print(5)
```

**Com um ciclo:**

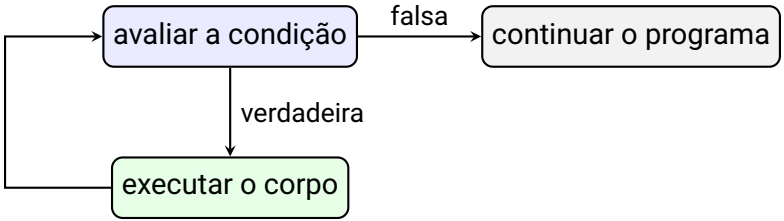
```
1 i = 1
2 while i <= 5:
3     print(i)
4     i = i + 1
```

O segundo programa descreve a regra de repetição. Alterar o limite deixa de exigir copiar ou remover instruções.

# Como Funciona um Ciclo `while`

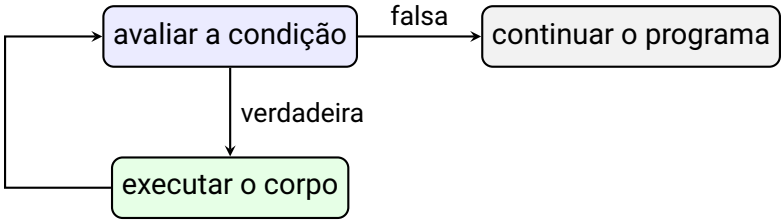


# Como Funciona um Ciclo `while`



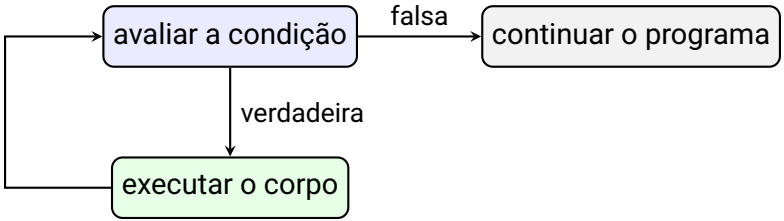

- A condição é avaliada **antes** de cada iteração.

# Como Funciona um Ciclo `while`




- A condição é avaliada **antes** de cada iteração.
- Se começar por ser falsa, o corpo não é executado nenhuma vez.

# Como Funciona um Ciclo `while`




- A condição é avaliada **antes** de cada iteração.
- Se começar por ser falsa, o corpo não é executado nenhuma vez.
- Depois do corpo, a execução regressa sempre à condição.

# Anatomia de um Ciclo `while`

```
1 i = 1                  # estado inicial
2 while i <= 5:          # condição de continuação
3     print(i)            # trabalho realizado
4     i = i + 1           # atualização do estado
```

# Anatomia de um Ciclo `while`

```
1 i = 1                  # estado inicial
2 while i <= 5:          # condição de continuação
3     print(i)            # trabalho realizado
4     i = i + 1           # atualização do estado
```

- `i` faz parte do **estado**: o seu valor pode mudar.

# Anatomia de um Ciclo `while`

```
1 i = 1                  # estado inicial
2 while i <= 5:          # condição de continuação
3     print(i)            # trabalho realizado
4     i = i + 1           # atualização do estado
```

- `i` faz parte do **estado**: o seu valor pode mudar.
- A condição decide se existe outra iteração.

# Anatomia de um Ciclo `while`

```
1 i = 1                  # estado inicial
2 while i <= 5:          # condição de continuação
3     print(i)            # trabalho realizado
4     i = i + 1           # atualização do estado
```

- `i` faz parte do **estado**: o seu valor pode mudar.
- A condição decide se existe outra iteração.
- O corpo realiza trabalho e atualiza o estado.

# Anatomia de um Ciclo `while`

```
1 i = 1                  # estado inicial
2 while i <= 5:          # condição de continuação
3     print(i)            # trabalho realizado
4     i = i + 1           # atualização do estado
```

- `i` faz parte do **estado**: o seu valor pode mudar.
- A condição decide se existe outra iteração.
- O corpo realiza trabalho e atualiza o estado.
- A atualização faz `i` aproximar-se da condição de paragem.

# Trace de um Primeiro Ciclo

```
1 i = 1
2 total = 0
3 while i <= 4:
4     total = total + i
5     i = i + 1
6 print(total)
```

| passo | `i` | `total` | condição |
|---|---|---|---|
| início | 1 | 0 | `True` |

# Trace de um Primeiro Ciclo

```
1 i = 1
2 total = 0
3 while i <= 4:
4     total = total + i
5     i = i + 1
6 print(total)
```

| passo | `i` | `total` | condição |
|---|---|---|---|
| início | 1 | 0 | `True` |
| 1 | 2 | 1 | `True` |

# Trace de um Primeiro Ciclo

```
1 i = 1
2 total = 0
3 while i <= 4:
4     total = total + i
5     i = i + 1
6 print(total)
```

| passo | `i` | `total` | condição |
|---|---|---|---|
| início | 1 | 0 | `True` |
| 1 | 2 | 1 | `True` |
| 2 | 3 | 3 | `True` |

# Trace de um Primeiro Ciclo

```
1 i = 1
2 total = 0
3 while i <= 4:
4     total = total + i
5     i = i + 1
6 print(total)
```

| passo | `i` | `total` | condição |
|---|---|---|---|
| início | 1 | 0 | `True` |
| 1 | 2 | 1 | `True` |
| 2 | 3 | 3 | `True` |
| 3 | 4 | 6 | `True` |

# Trace de um Primeiro Ciclo

```
1 i = 1
2 total = 0
3 while i <= 4:
4     total = total + i
5     i = i + 1
6 print(total)
```

| passo | `i` | `total` | condição |
|---|---|---|---|
| início | 1 | 0 | `True` |
| 1 | 2 | 1 | `True` |
| 2 | 3 | 3 | `True` |
| 3 | 4 | 6 | `True` |
| 4 | 5 | 10 | `False` |

# Trace de um Primeiro Ciclo

```
1 i = 1
2 total = 0
3 while i <= 4:
4     total = total + i
5     i = i + 1
6 print(total)
```

| passo | `i` | `total` | condição |
|---|---|---|---|
| início | 1 | 0 | `True` |
| 1 | 2 | 1 | `True` |
| 2 | 3 | 3 | `True` |
| 3 | 4 | 6 | `True` |
| 4 | 5 | 10 | `False` |

Quando o ciclo termina, `total` contém $1 + 2 + 3 + 4$; o programa imprime `10`.

# Previsão: O Que é Impresso?

Sem executar o programa, determine os valores apresentados no ecrã e o valor final de `x`.

```
1 x = 1
2 while x < 20:
3     print(x)
4     x = x * 2
```

A justificação resulta de acompanhar o valor de `x`, o resultado da condição e o valor impresso em cada iteração.

# Resultado Esperado: Potências de Dois

**Problema:** determinar o que o ciclo anterior imprime e com que valor termina `x`.

- Valores impressos: `1`, `2`, `4`, `8`, `16`.

# Resultado Esperado: Potências de Dois

**Problema:** determinar o que o ciclo anterior imprime e com que valor termina `x`.

- Valores impressos: `1`, `2`, `4`, `8`, `16`.
- Depois da última atualização, `x` vale `32`.

# Resultado Esperado: Potências de Dois

**Problema:** determinar o que o ciclo anterior imprime e com que valor termina `x`.

- Valores impressos: `1`, `2`, `4`, `8`, `16`.
- Depois da última atualização, `x` vale `32`.
- A condição `x < 20` é então falsa e o ciclo termina.

# Estado, Progresso e Resultado

Ao desenhar um ciclo, responda a quatro perguntas:

1. **Estado inicial:** que valores existem antes da primeira iteração?

# Estado, Progresso e Resultado

Ao desenhar um ciclo, responda a quatro perguntas:

1. **Estado inicial:** que valores existem antes da primeira iteração?
2. **Condição:** enquanto é que faz sentido continuar?

# Estado, Progresso e Resultado

Ao desenhar um ciclo, responda a quatro perguntas:

1. **Estado inicial:** que valores existem antes da primeira iteração?
2. **Condição:** enquanto é que faz sentido continuar?
3. **Atualização:** o que muda em cada iteração?

# Estado, Progresso e Resultado

Ao desenhar um ciclo, responda a quatro perguntas:

1. **Estado inicial:** que valores existem antes da primeira iteração?
2. **Condição:** enquanto é que faz sentido continuar?
3. **Atualização:** o que muda em cada iteração?
4. **Resultado:** que significado têm os valores quando o ciclo termina?

# Estado, Progresso e Resultado

Ao desenhar um ciclo, responda a quatro perguntas:

1. **Estado inicial:** que valores existem antes da primeira iteração?
2. **Condição:** enquanto é que faz sentido continuar?
3. **Atualização:** o que muda em cada iteração?
4. **Resultado:** que significado têm os valores quando o ciclo termina?

**Regra prática**

Se a atualização não fizer progresso em direção à condição falsa, o ciclo pode nunca terminar.

# Quando um Ciclo Não Termina

**Sem atualização:**

```
1 i = 1
2 while i <= 5:
3     print(i)
```

**Atualização na direção errada:**

```
1 i = 1
2 while i <= 5:
3     print(i)
4     i = i - 1
```

# Quando um Ciclo Não Termina

**Sem atualização:**

```
1 i = 1
2 while i <= 5:
3     print(i)
```

**Atualização na direção errada:**

```
1 i = 1
2 while i <= 5:
3     print(i)
4     i = i - 1
```

Nos dois casos, a condição permanece verdadeira. Para justificar a terminação, identifique uma quantidade não negativa que diminui em cada iteração.

# Outline

1. Introdução à repetição
2. Contadores e acumuladores
3. Ciclos controlados pelos valores
4. Fibonacci como exemplo de estado
5. Raciocinar e testar ciclos

# Contar de 1 até n

```
1 def mostrar_ate(n: int) -> None:
2     i = 1
3     while i <= n:
4         print(i)
5         i = i + 1
```

# Contar de 1 até n

```
1 def mostrar_ate(n: int) -> None:
2     i = 1
3     while i <= n:
4         print(i)
5         i = i + 1
```

- `i` é um **contador**: identifica a iteração atual.

# Contar de 1 até n

```
1 def mostrar_ate(n: int) -> None:
2     i = 1
3     while i <= n:
4         print(i)
5         i = i + 1
```

- `i` é um **contador**: identifica a iteração atual.
- Antes de cada iteração, já foram impressos os valores de `1` a `i - 1`.

# Contar de 1 até n

```
1 def mostrar_ate(n: int) -> None:
2     i = 1
3     while i <= n:
4         print(i)
5         i = i + 1
```

- `i` é um **contador**: identifica a iteração atual.
- Antes de cada iteração, já foram impressos os valores de `1` a `i - 1`.
- Quando termina, `i == n + 1`; todos os valores pretendidos foram impressos.

# Fronteiras: < ou <=?

```
1 i = 1
2 while i < n:
3     print(i)
4     i = i + 1
```

Imprime de `1` até `n - 1`.

# Fronteiras: < ou <=?

```
1 i = 1
2 while i < n:
3     print(i)
4     i = i + 1
```

Imprime de `1` até `n - 1`.

```
1 i = 1
2 while i <= n:
3     print(i)
4     i = i + 1
```

Imprime de `1` até `n`.

Testar `n = 0`, `n = 1` e `n = 2` ajuda a detetar erros de fronteira.

# Acumular um Resultado

```
1 def soma_ate(n: int) -> int:
2     i = 1
3     total = 0
4     while i <= n:
5         total = total + i
6         i = i + 1
7     return total
```

# Acumular um Resultado

```
1 def soma_ate(n: int) -> int:
2     i = 1
3     total = 0
4     while i <= n:
5         total = total + i
6         i = i + 1
7     return total
```

- `i` controla quantos valores já foram processados.

# Acumular um Resultado

```
1 def soma_ate(n: int) -> int:
2     i = 1
3     total = 0
4     while i <= n:
5         total = total + i
6         i = i + 1
7     return total
```

- `i` controla quantos valores já foram processados.
- `total` é um **acumulador**: guarda o resultado parcial.

# Acumular um Resultado

```python
def soma_ate(n: int) -> int:
    i = 1
    total = 0
    while i <= n:
        total = total + i
        i = i + 1
    return total
```

- `i` controla quantos valores já foram processados.
- `total` é um **acumulador**: guarda o resultado parcial.
- O valor inicial `0` é adequado porque não altera uma soma.

# Trace da Acumulação

Para `soma_ate(4)`, no início de cada iteração:

| iteração | `i` | `total` | significado de `total` |
|---|---|---|---|
| 1 | 1 | 0 | soma dos valores anteriores a 1 |

# Trace da Acumulação

Para `soma_ate(4)`, no início de cada iteração:

| iteração | `i` | `total` | significado de `total` |
|---|---|---|---|
| 1 | 1 | 0 | soma dos valores anteriores a 1 |
| 2 | 2 | 1 | soma dos valores anteriores a 2 |

# Trace da Acumulação

Para `soma_ate(4)`, no início de cada iteração:

| iteração | `i` | `total` | significado de `total` |
|---|---|---|---|
| 1 | 1 | 0 | soma dos valores anteriores a 1 |
| 2 | 2 | 1 | soma dos valores anteriores a 2 |
| 3 | 3 | 3 | soma dos valores anteriores a 3 |

# Trace da Acumulação

Para `soma_ate(4)`, no início de cada iteração:

| iteração | `i` | `total` | significado de `total` |
|---|---|---|---|
| 1 | 1 | 0 | soma dos valores anteriores a 1 |
| 2 | 2 | 1 | soma dos valores anteriores a 2 |
| 3 | 3 | 3 | soma dos valores anteriores a 3 |
| 4 | 4 | 6 | soma dos valores anteriores a 4 |

# Trace da Acumulação

Para `soma_ate(4)`, no início de cada iteração:

| iteração | `i` | `total` | significado de `total` |
|---|---|---|---|
| 1 | 1 | 0 | soma dos valores anteriores a 1 |
| 2 | 2 | 1 | soma dos valores anteriores a 2 |
| 3 | 3 | 3 | soma dos valores anteriores a 3 |
| 4 | 4 | 6 | soma dos valores anteriores a 4 |
| fim | 5 | 10 | soma dos valores de 1 a 4 |

# Questão: Como Somar Só os Pares?

Queremos adaptar `soma_ate(n)` para somar apenas os valores pares entre `2` e `n`. Para `n == 7`, o resultado deve ser `12`.

```
1 def soma_pares(n: int) -> int:
2     par = ...
3     total = ...
4     while ...:
5         total = ...
6         par = ...
7     return total
```

Que valores iniciais, condição e atualizações completam a ideia? É necessário testar também os números ímpares?

# Uma Solução Possível: Somar os Pares

**Problema:** somar os inteiros pares entre `2` e `n`, inclusive, com um ciclo **`while`**.

```
1 def soma_pares(n: int) -> int:
2     par = 2
3     total = 0
4     while par <= n:
5         total = total + par
6         par = par + 2
7     return total
```

O contador percorre apenas candidatos relevantes: `2, 4, 6, ...`.

# Produto Acumulado: Factorial

Para `n >= 0`, $n! = 1 \times 2 \times \cdots \times n$ e $0! = 1$.

```
1 def factorial(n: int) -> int:
2     i = 1
3     produto = 1
4     while i <= n:
5         produto = produto * i
6         i = i + 1
7     return produto
```

# Produto Acumulado: Factorial

Para `n >= 0`, $n! = 1 \times 2 \times \cdots \times n$ e $0! = 1$.

```
1 def factorial(n: int) -> int:
2     i = 1
3     produto = 1
4     while i <= n:
5         produto = produto * i
6         i = i + 1
7     return produto
```

O acumulador começa em `1`, o elemento neutro da multiplicação. Para `n == 0`, o corpo não é executado e o resultado é corretamente `1`.

# Dois Papéis no Estado do Ciclo

## Contador

Indica onde estamos ou quantos valores foram processados.
Exemplos: `i`, `par`, número de passos.

## Acumulador

Resume o trabalho realizado até ao momento.
Exemplos: `total`, `produto`, melhor valor encontrado.

# Dois Papéis no Estado do Ciclo

**Contador**

Indica onde estamos ou quantos valores foram processados.
Exemplos: `i`, `par`, número de passos.

**Acumulador**

Resume o trabalho realizado até ao momento.
Exemplos: `total`, `produto`, melhor valor encontrado.

Uma variável pode ter outro papel, mas cada nome deve ter um significado claro durante toda a execução.

# Outline

1. Introdução à repetição
2. Contadores e acumuladores
3. Ciclos controlados pelos valores
4. Fibonacci como exemplo de estado
5. Raciocinar e testar ciclos

# Quantas Iterações Serão Necessárias?

Nem sempre conhecemos antecipadamente o número de iterações. Para contar os algarismos de um inteiro positivo:

```
1 def numero_algarismos(n: int) -> int:
2     quantidade = 0
3     while n > 0:
4         n = n // 10
5         quantidade = quantidade + 1
6     return quantidade
```

# Quantas Iterações Serão Necessárias?

Nem sempre conhecemos antecipadamente o número de iterações. Para contar os algarismos de um inteiro positivo:

```
1 def numero_algarismos(n: int) -> int:
2     quantidade = 0
3     while n > 0:
4         n = n // 10
5         quantidade = quantidade + 1
6     return quantidade
```

O quociente `n` perde um algarismo em cada iteração; o ciclo termina quando já não resta nenhum.

# Trace: Remover um Algarismo de Cada Vez

Para `numero_algarismos(5072)`:

| iteração | `n` antes | `n // 10` | `quantidade` depois |
|---|---|---|---|
| 1 | 5072 | 507 | 1 |

# Trace: Remover um Algarismo de Cada Vez

Para `numero_algarismos(5072)`:

| iteração | `n` antes | `n // 10` | `quantidade` depois |
|---|---|---|---|
| 1 | 5072 | 507 | 1 |
| 2 | 507 | 50 | 2 |

# Trace: Remover um Algarismo de Cada Vez

Para `numero_algarismos(5072)`:

| iteração | `n` antes | `n // 10` | `quantidade` depois |
|---|---|---|---|
| 1 | 5072 | 507 | 1 |
| 2 | 507 | 50 | 2 |
| 3 | 50 | 5 | 3 |

# Trace: Remover um Algarismo de Cada Vez

Para `numero_algarismos(5072)`:

| iteração | `n` antes | `n // 10` | `quantidade` depois |
|---|---|---|---|
| 1 | 5072 | 507 | 1 |
| 2 | 507 | 50 | 2 |
| 3 | 50 | 5 | 3 |
| 4 | 5 | 0 | 4 |

# Trace: Remover um Algarismo de Cada Vez

Para `numero_algarismos(5072)`:

| iteração | `n` antes | `n // 10` | `quantidade` depois |
|---|---|---|---|
| 1 | 5072 | 507 | 1 |
| 2 | 507 | 50 | 2 |
| 3 | 50 | 5 | 3 |
| 4 | 5 | 0 | 4 |

O valor `n` é simultaneamente dado de trabalho e medida de progresso: diminui até zero.

# Construção: Somar os Algarismos

Para somar os algarismos de `5072`, podemos usar repetidamente:

- `n % 10` para obter o último algarismo;
- `n // 10` para remover o último algarismo.
- Que valor deve acumular a soma parcial?
- Qual deve ser a condição de continuação?
- Que valor diminui em cada iteração e garante progresso?

O resultado esperado para `5072` é `14`.

# Uma Solução Possível: Soma dos Algarismos

**Problema:** somar os algarismos de `n >= 0`, removendo um algarismo em cada iteração.

```
1 def soma_algarismos(n: int) -> int:
2     total = 0
3     while n > 0:
4         total = total + n % 10
5         n = n // 10
6     return total
```

- `total` acumula os algarismos já removidos.
- `n` diminui até zero e garante a terminação.

# Procurar um Divisor

O ciclo pode terminar porque esgotámos os candidatos ou porque encontrámos uma resposta:

```
1 def menor_divisor(n: int) -> int:
2     d = 2
3     encontrado = False
4     while d < n and not encontrado:
5         if n % d == 0:
6             encontrado = True
7         else:
8             d = d + 1
9     return d if encontrado else n
```

# Procurar um Divisor

O ciclo pode terminar porque esgotámos os candidatos ou porque encontrámos uma resposta:

```
1 def menor_divisor(n: int) -> int:
2     d = 2
3     encontrado = False
4     while d < n and not encontrado:
5         if n % d == 0:
6             encontrado = True
7         else:
8             d = d + 1
9     return d if encontrado else n
```

Para `n > 1`, a função devolve o menor divisor maior ou igual a 2. Se não encontrar outro, devolve o próprio `n`.

# Testar se um Número é Primo

Um inteiro `n > 1` é primo quando o seu menor divisor é ele próprio:

```
1 def e_primo(n: int) -> bool:
2     if n <= 1:
3         return False
4     return menor_divisor(n) == n
5
6 assert e_primo(2)
7 assert e_primo(13)
8 assert not e_primo(1)
9 assert not e_primo(15)
```

# Testar se um Número é Primo

Um inteiro `n > 1` é primo quando o seu menor divisor é ele próprio:

```python
def e_primo(n: int) -> bool:
    if n <= 1:
        return False
    return menor_divisor(n) == n

assert e_primo(2)
assert e_primo(13)
assert not e_primo(1)
assert not e_primo(15)
```

Este exemplo combina alternativas, funções e um ciclo sem introduzir ainda coleções de valores.

# Outline

1. Introdução à repetição
2. Contadores e acumuladores
3. Ciclos controlados pelos valores
4. Fibonacci como exemplo de estado
5. Raciocinar e testar ciclos

# A Sequência de Fibonacci

A sequência é definida por:

$$F_0 = 0, \qquad F_1 = 1, \qquad F_{n+2} = F_n + F_{n+1}.$$

# A Sequência de Fibonacci

A sequência é definida por:

$$F_0 = 0, \qquad F_1 = 1, \qquad F_{n+2} = F_n + F_{n+1}.$$

Os primeiros valores são:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21, \ldots$$

# A Sequência de Fibonacci

A sequência é definida por:

$$F_0 = 0, \qquad F_1 = 1, \qquad F_{n+2} = F_n + F_{n+1}.$$

Os primeiros valores são:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21, \ldots$$

Para calcular o valor seguinte basta manter os dois valores consecutivos mais recentes; não é necessário guardar toda a sequência.

# Que Estado Devemos Manter?

No início da iteração de índice `i`, pretendemos que:

$$\mathsf{a} = F_i \qquad \text{e} \qquad \mathsf{b} = F_{i+1}.$$

# Que Estado Devemos Manter?

No início da iteração de índice `i`, pretendemos que:

$$\texttt{a} = F_i \qquad \text{e} \qquad \texttt{b} = F_{i+1}.$$

- Inicialmente, `i = 0`, `a = 0` e `b = 1`.

# Que Estado Devemos Manter?

No início da iteração de índice `i`, pretendemos que:

$$\texttt{a} = F_i \qquad \text{e} \qquad \texttt{b} = F_{i+1}.$$

- Inicialmente, `i = 0`, `a = 0` e `b = 1`.
- Para avançar um índice, o novo par deve ser $(F_{i+1}, F_i + F_{i+1})$.

# Que Estado Devemos Manter?

No início da iteração de índice `i`, pretendemos que:

$$\texttt{a} = F_i \qquad \text{e} \qquad \texttt{b} = F_{i+1}.$$

- Inicialmente, `i = 0`, `a = 0` e `b = 1`.
- Para avançar um índice, o novo par deve ser $(F_{i+1}, F_i + F_{i+1})$.
- A afetação paralela `a, b = b, a + b` faz essa atualização.

# Que Estado Devemos Manter?

No início da iteração de índice `i`, pretendemos que:

$$\texttt{a} = F_i \qquad \text{e} \qquad \texttt{b} = F_{i+1}.$$

- Inicialmente, `i = 0`, `a = 0` e `b = 1`.
- Para avançar um índice, o novo par deve ser $(F_{i+1}, F_i + F_{i+1})$.
- A afetação paralela `a, b = b, a + b` faz essa atualização.
- Quando `i == n`, `a` é precisamente $F_n$.

# Fibonacci com um Ciclo `while`

```
1 def fibonacci(n: int) -> int:
2     """Devolve F_n, para n >= 0."""
3     i = 0
4     a, b = 0, 1
5     while i < n:
6         a, b = b, a + b
7         i = i + 1
8     return a
```

# Fibonacci com um Ciclo `while`

```
1 def fibonacci(n: int) -> int:
2     """Devolve F_n, para n >= 0."""
3     i = 0
4     a, b = 0, 1
5     while i < n:
6         a, b = b, a + b
7         i = i + 1
8     return a
```

- `i` é o contador.

# Fibonacci com um Ciclo `while`

```
1 def fibonacci(n: int) -> int:
2     """Devolve F_n, para n >= 0."""
3     i = 0
4     a, b = 0, 1
5     while i < n:
6         a, b = b, a + b
7         i = i + 1
8     return a
```

- `i` é o contador.
- `a` e `b` são o estado necessário para o cálculo.

# Fibonacci com um Ciclo `while`

```
1 def fibonacci(n: int) -> int:
2     """Devolve F_n, para n >= 0."""
3     i = 0
4     a, b = 0, 1
5     while i < n:
6         a, b = b, a + b
7         i = i + 1
8     return a
```

- `i` é o contador.
- `a` e `b` são o estado necessário para o cálculo.
- `n - i` diminui uma unidade em cada iteração.

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |
| após 1 iteração | 1 | 1 | 1 |

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |
| após 1 iteração | 1 | 1 | 1 |
| após 2 iterações | 2 | 1 | 2 |

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |
| após 1 iteração | 1 | 1 | 1 |
| após 2 iterações | 2 | 1 | 2 |
| após 3 iterações | 3 | 2 | 3 |

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |
| após 1 iteração | 1 | 1 | 1 |
| após 2 iterações | 2 | 1 | 2 |
| após 3 iterações | 3 | 2 | 3 |
| após 4 iterações | 4 | 3 | 5 |

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |
| após 1 iteração | 1 | 1 | 1 |
| após 2 iterações | 2 | 1 | 2 |
| após 3 iterações | 3 | 2 | 3 |
| após 4 iterações | 4 | 3 | 5 |
| após 5 iterações | 5 | 5 | 8 |

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |
| após 1 iteração | 1 | 1 | 1 |
| após 2 iterações | 2 | 1 | 2 |
| após 3 iterações | 3 | 2 | 3 |
| após 4 iterações | 4 | 3 | 5 |
| após 5 iterações | 5 | 5 | 8 |
| após 6 iterações | 6 | 8 | 13 |

# Trace de `fibonacci(6)`

| momento | `i` | `a` | `b` |
|---|---|---|---|
| inicial | 0 | 0 | 1 |
| após 1 iteração | 1 | 1 | 1 |
| após 2 iterações | 2 | 1 | 2 |
| após 3 iterações | 3 | 2 | 3 |
| após 4 iterações | 4 | 3 | 5 |
| após 5 iterações | 5 | 5 | 8 |
| após 6 iterações | 6 | 8 | 13 |

O ciclo termina com `i == 6`; a função devolve `a == 8`.

# Previsão: O Que Devolve a Variante?

Considere a função seguinte:

```
1 def fibonacci_seguinte(n: int) -> int:
2     i = 0
3     a, b = 0, 1
4     while i < n:
5         a, b = b, a + b
6         i = i + 1
7     return b
```

Determine o valor obtido para `n == 0`, `n == 1` e `n == 5`. Que relação existe entre o resultado e $F_n$?

# Resultado Esperado: Fibonacci Seguinte

**Problema:** explicar o resultado da variante que devolve `b` em vez de `a`.

- Para `n == 0`, devolve `1` $= F_1$.

# Resultado Esperado: Fibonacci Seguinte

**Problema:** explicar o resultado da variante que devolve `b` em vez de `a`.

- Para `n == 0`, devolve `1` $= F_1$.
- Para `n == 1`, devolve `1` $= F_2$.

# Resultado Esperado: Fibonacci Seguinte

**Problema:** explicar o resultado da variante que devolve `b` em vez de `a`.

- Para `n == 0`, devolve `1` $= F_1$.
- Para `n == 1`, devolve `1` $= F_2$.
- Para `n == 5`, devolve `8` $= F_6$.

# Resultado Esperado: Fibonacci Seguinte

**Problema:** explicar o resultado da variante que devolve `b` em vez de `a`.

- Para `n == 0`, devolve `1` $= F_1$.
- Para `n == 1`, devolve `1` $= F_2$.
- Para `n == 5`, devolve `8` $= F_6$.
- Em geral, quando o ciclo termina, `b` contém $F_{n+1}$.

# Outline

1. Introdução à repetição
2. Contadores e acumuladores
3. Ciclos controlados pelos valores
4. Fibonacci como exemplo de estado
5. Raciocinar e testar ciclos

# Erros Frequentes

- **Não inicializar** uma variável usada na condição ou no corpo.

# Erros Frequentes

- **Não inicializar** uma variável usada na condição ou no corpo.
- **Não atualizar** o estado que controla a condição.

# Erros Frequentes

- **Não inicializar** uma variável usada na condição ou no corpo.
- **Não atualizar** o estado que controla a condição.
- Atualizar **antes** de usar o valor quando a ordem devia ser a inversa.

# Erros Frequentes

- **Não inicializar** uma variável usada na condição ou no corpo.
- **Não atualizar** o estado que controla a condição.
- Atualizar **antes** de usar o valor quando a ordem devia ser a inversa.
- Escolher `<` quando o limite devia ser incluído, ou `<=` quando devia ser excluído.

# Erros Frequentes

- **Não inicializar** uma variável usada na condição ou no corpo.
- **Não atualizar** o estado que controla a condição.
- Atualizar **antes** de usar o valor quando a ordem devia ser a inversa.
- Escolher `<` quando o limite devia ser incluído, ou `<=` quando devia ser excluído.
- Começar um acumulador com o valor errado: por exemplo, produto em `0` em vez de `1`.

# Erros Frequentes

- **Não inicializar** uma variável usada na condição ou no corpo.
- **Não atualizar** o estado que controla a condição.
- Atualizar **antes** de usar o valor quando a ordem devia ser a inversa.
- Escolher `<` quando o limite devia ser incluído, ou `<=` quando devia ser excluído.
- Começar um acumulador com o valor errado: por exemplo, produto em `0` em vez de `1`.

**Diagnóstico**

Um trace curto permite verificar separadamente o estado inicial, a condição, o corpo e a atualização.

# Uma Primeira Noção de Invariante

Um **invariante** descreve o significado do estado sempre que a condição do ciclo é avaliada.

# Uma Primeira Noção de Invariante

Um **invariante** descreve o significado do estado sempre que a condição do ciclo é avaliada.

- Em `soma_ate`, `total` é a soma de `1` até `i - 1`.

O invariante ajuda a escolher a inicialização, compreender a atualização e explicar por que o resultado final está correto.

# Uma Primeira Noção de Invariante

Um **invariante** descreve o significado do estado sempre que a condição do ciclo é avaliada.

- Em `soma_ate`, `total` é a soma de `1` até `i - 1`.
- Em `numero_algarismos`, `quantidade` conta os algarismos já removidos.

O invariante ajuda a escolher a inicialização, compreender a atualização e explicar por que o resultado final está correto.

# Uma Primeira Noção de Invariante

Um **invariante** descreve o significado do estado sempre que a condição do ciclo é avaliada.

- Em `soma_ate`, `total` é a soma de `1` até `i - 1`.
- Em `numero_algarismos`, `quantidade` conta os algarismos já removidos.
- Em `fibonacci`, `a == F_i` e `b == F_(i+1)`.

O invariante ajuda a escolher a inicialização, compreender a atualização e explicar por que o resultado final está correto.

# Como Desenhar um Ciclo

1. Escreva o que deve significar cada variável.

# Como Desenhar um Ciclo

1. Escreva o que deve significar cada variável.
2. Escolha um estado inicial em que esses significados sejam verdadeiros.

# Como Desenhar um Ciclo

1. Escreva o que deve significar cada variável.
2. Escolha um estado inicial em que esses significados sejam verdadeiros.
3. Escreva a condição que distingue trabalho por fazer de trabalho concluído.

# Como Desenhar um Ciclo

1. Escreva o que deve significar cada variável.
2. Escolha um estado inicial em que esses significados sejam verdadeiros.
3. Escreva a condição que distingue trabalho por fazer de trabalho concluído.
4. Atualize o estado preservando o seu significado.

# Como Desenhar um Ciclo

1. Escreva o que deve significar cada variável.
2. Escolha um estado inicial em que esses significados sejam verdadeiros.
3. Escreva a condição que distingue trabalho por fazer de trabalho concluído.
4. Atualize o estado preservando o seu significado.
5. Identifique uma quantidade que progride para justificar a terminação.

# Como Desenhar um Ciclo

1. Escreva o que deve significar cada variável.
2. Escolha um estado inicial em que esses significados sejam verdadeiros.
3. Escreva a condição que distingue trabalho por fazer de trabalho concluído.
4. Atualize o estado preservando o seu significado.
5. Identifique uma quantidade que progride para justificar a terminação.
6. Relacione a condição falsa com o resultado a devolver.

# Testar Funções com Ciclos

Inclua casos que exercitem quantidades diferentes de iterações:

```
1 assert soma_ate(0) == 0       # zero iterações
2 assert soma_ate(1) == 1       # uma iteração
3 assert soma_ate(5) == 15      # várias iterações
4
5 assert fibonacci(0) == 0
6 assert fibonacci(1) == 1
7 assert fibonacci(7) == 13
```

# Testar Funções com Ciclos

Inclua casos que exercitem quantidades diferentes de iterações:

```
1 assert soma_ate(0) == 0       # zero iterações
2 assert soma_ate(1) == 1       # uma iteração
3 assert soma_ate(5) == 15      # várias iterações
4
5 assert fibonacci(0) == 0
6 assert fibonacci(1) == 1
7 assert fibonacci(7) == 13
```

Casos pequenos permitem confirmar o resultado à mão; valores de fronteira expõem condições e inicializações incorretas.

# Questão: Como Inverter a Contagem?

No primeiro ciclo da aula contámos de `1` até um limite. O que deve mudar para imprimir de `n` até `0`?

- Com que valor começa o contador?
- Enquanto que condição é verdadeira?
- Como muda o contador em cada iteração?
- Com que valor termina?

# Uma Solução Possível: Contagem Decrescente

**Problema:** imprimir os inteiros de `n` até `0` com um ciclo **`while`**.

```
1 def contagem_decrescente(n: int) -> None:
2     atual = n
3     while atual >= 0:
4         print(atual)
5         atual = atual - 1
```

Quando termina, `atual == -1`; a distância até `-1` diminui em cada iteração.

# MDC por Restos Sucessivos

Para inteiros positivos, o algoritmo de Euclides usa a propriedade

$$\mathsf{mdc}(a, b) = \mathsf{mdc}(b, a \bmod b).$$

# MDC por Restos Sucessivos

Para inteiros positivos, o algoritmo de Euclides usa a propriedade

$$\text{mdc}(a, b) = \text{mdc}(b, a \bmod b).$$

- Substituímos o par por `(b, a % b)`.

# MDC por Restos Sucessivos

Para inteiros positivos, o algoritmo de Euclides usa a propriedade

$$\mathsf{mdc}(a, b) = \mathsf{mdc}(b, a \bmod b).$$

- Substituímos o par por `(b, a % b)`.
- O resto é menor do que `b` e acaba por chegar a zero.

# MDC por Restos Sucessivos

Para inteiros positivos, o algoritmo de Euclides usa a propriedade

$$\mathsf{mdc}(a, b) = \mathsf{mdc}(b, a \bmod b).$$

- Substituímos o par por `(b, a % b)`.
- O resto é menor do que `b` e acaba por chegar a zero.
- Quando `b == 0`, o valor de `a` é o MDC inicial.

# Implementar o Algoritmo de Euclides

```python
def mdc(a: int, b: int) -> int:
    while b != 0:
        a, b = b, a % b
    return a

assert mdc(48, 36) == 12
assert mdc(100, 75) == 25
```

# Implementar o Algoritmo de Euclides

```
1 def mdc(a: int, b: int) -> int:
2     while b != 0:
3         a, b = b, a % b
4     return a
5
6 assert mdc(48, 36) == 12
7 assert mdc(100, 75) == 25
```

A afetação paralela é essencial: o novo resto é calculado com os dois valores anteriores antes de `a` e `b` serem alterados.

# Trace do Algoritmo de Euclides

| passo | a | b | a % b |
|---|---|---|---|
| inicial | 48 | 36 | 12 |

# Trace do Algoritmo de Euclides

| passo | a | b | a % b |
|---|---|---|---|
| inicial | 48 | 36 | 12 |
| 1 | 36 | 12 | 0 |

# Trace do Algoritmo de Euclides

| passo | a | b | a % b |
|---|---|---|---|
| inicial | 48 | 36 | 12 |
| 1 | 36 | 12 | 0 |
| 2 | 12 | 0 | — |

# Trace do Algoritmo de Euclides

| passo | a | b | a % b |
|---|---|---|---|
| inicial | 48 | 36 | 12 |
| 1 | 36 | 12 | 0 |
| 2 | 12 | 0 | — |

O segundo componente diminui até zero; o valor final de `a` é `12`.

# A Regra de Collatz

Partindo de um inteiro positivo `n`:

# A Regra de Collatz

Partindo de um inteiro positivo `n`:

- se for par, substituir `n` por `n // 2`;

# A Regra de Collatz

Partindo de um inteiro positivo `n`:

- se for par, substituir `n` por `n // 2`;
- se for ímpar, substituir `n` por `3 * n + 1`;

# A Regra de Collatz

Partindo de um inteiro positivo `n`:

- se for par, substituir `n` por `n // 2`;
- se for ímpar, substituir `n` por `3 * n + 1`;
- repetir enquanto `n != 1`.

# A Regra de Collatz

Partindo de um inteiro positivo `n`:

- se for par, substituir `n` por `n // 2`;
- se for ímpar, substituir `n` por `3 * n + 1`;
- repetir enquanto `n != 1`.

Para `n == 6`: $6, 3, 10, 5, 16, 8, 4, 2, 1$.

# Contar Passos de Collatz

```
1  def passos_collatz(n: int) -> int:
2      passos = 0
3      while n != 1:
4          if n % 2 == 0:
5              n = n // 2
6          else:
7              n = 3 * n + 1
8          passos = passos + 1
9      return passos
10
11 assert passos_collatz(6) == 8
```

# Collatz e Terminação

O programa termina para todos os valores já testados, mas não se conhece uma prova de que chegue sempre a `1` para qualquer inteiro positivo.

# Collatz e Terminação

O programa termina para todos os valores já testados, mas não se conhece uma prova de que chegue sempre a `1` para qualquer inteiro positivo.

- O estado muda em cada iteração.

# Collatz e Terminação

O programa termina para todos os valores já testados, mas não se conhece uma prova de que chegue sempre a `1` para qualquer inteiro positivo.

- O estado muda em cada iteração.
- Ao contrário dos exemplos anteriores, não conhecemos uma medida simples que diminua sempre.

# Collatz e Terminação

O programa termina para todos os valores já testados, mas não se conhece uma prova de que chegue sempre a `1` para qualquer inteiro positivo.

- O estado muda em cada iteração.
- Ao contrário dos exemplos anteriores, não conhecemos uma medida simples que diminua sempre.
- Testar muitos casos não substitui uma justificação geral de terminação.

# Aproximar uma Raiz Quadrada

Para `n > 0`, mantemos um intervalo `[baixo, alto]` que contém $\sqrt{n}$.

# Aproximar uma Raiz Quadrada

Para `n > 0`, mantemos um intervalo `[baixo, alto]` que contém $\sqrt{n}$.

- Calcular o ponto médio.

# Aproximar uma Raiz Quadrada

Para `n > 0`, mantemos um intervalo `[baixo, alto]` que contém $\sqrt{n}$.

- Calcular o ponto médio.
- Se o quadrado do ponto médio for menor do que `n`, subir o limite inferior; caso contrário, baixar o limite superior.

# Aproximar uma Raiz Quadrada

Para `n > 0`, mantemos um intervalo `[baixo, alto]` que contém $\sqrt{n}$.

- Calcular o ponto médio.
- Se o quadrado do ponto médio for menor do que `n`, subir o limite inferior; caso contrário, baixar o limite superior.
- Repetir enquanto a largura do intervalo for superior à tolerância.

# Bisseção para a Raiz Quadrada

```python
def raiz_bissecao(n: float, eps: float) -> float:
    baixo = 0.0
    alto = max(1.0, n)
    while alto - baixo > eps:
        meio = (baixo + alto) / 2
        if meio * meio < n:
            baixo = meio
        else:
            alto = meio
    return (baixo + alto) / 2
```

# Bisseção para a Raiz Quadrada

```python
def raiz_bissecao(n: float, eps: float) -> float:
    baixo = 0.0
    alto = max(1.0, n)
    while alto - baixo > eps:
        meio = (baixo + alto) / 2
        if meio * meio < n:
            baixo = meio
        else:
            alto = meio
    return (baixo + alto) / 2
```

O intervalo contém sempre a solução e a sua largura é dividida por dois em cada iteração.

# Conclusão

## Hoje vimos

- execução e construção de ciclos `while`
- contadores, acumuladores e condições de paragem
- estado e invariantes em algoritmos numéricos

## A seguir

- decompor problemas em funções pequenas
- especificar responsabilidades e contratos informais
- organizar e testar programas com várias funções